

Relatório

Aluno: Mateus Crespo

Disciplina: INF01087 – Introdução à Ciência da Computação

Professor: Mateus Grellert

Data de entrega: 13/06/26

Parte 1 – Escolha da Ferramenta:

- Nome da ferramenta: Gemini
- Data de uso: 12/06/2026
- Versão (gratuita ou paga): gratuita
- Acesso à internet: Sim

Parte 2 - Explicação Conceitual:

- **Prompt simples:** O que é recursão em programação?
- **Resposta Prompt simples:**

Recursão em programação é uma técnica onde uma **função chama a si mesma** para resolver um problema.

A ideia central é pegar um problema grande e complexo e dividi-lo em pedaços menores e idênticos. Em vez de usar uma estrutura de repetição tradicional (como `for` ou `while`), a função vai se "autoinvocando" até chegar a um resultado.

Pense na recursão como aquelas bonecas russas (Matrioscas): para chegar na menor boneca possível, você precisa abrir uma boneca maior, que contém outra menor, e assim por diante.

Os Dois Pilares da Recursão

Para que uma função recursiva funcione corretamente e não trave o computador em um loop infinito, ela obrigatoriamente precisa de duas coisas:

1. **Caso Base (Condição de Parada):** É a condição que diz quando a função deve parar de chamar a si mesma. É o menor subproblema possível, que tem uma resposta direta.
2. **Caso Recursivo:** É a parte da função onde ela chama a si mesma, mas sempre modificando o argumento para que ele se aproxime, passo a passo, do caso base.

- **Prompt elaborado:** Explique o conceito de recursão em programação para um estudante do primeiro ano de Ciência da Computação da UFRGS, que já programa em C, mas nunca usou recursão antes. Use linguagem simples e didática, e finalize com um resumo de até 3 frases resumindo o conteúdo.
- **Resposta Prompt elaborado:**

O que é Recursão?

Em termos simples, a recursão acontece quando **uma função chama a si mesma** para resolver um problema.

Imagine que você precisa resolver um problema grande. Em vez de tentar resolver tudo de uma vez com um laço (while ou for), você divide o problema em uma parte menor e passa essa parte para a própria função resolver. Ela vai se dividindo até chegar em uma resposta óbvia.

A Regra de Ouro da Recursão

Toda função recursiva **obrigatoriamente** precisa de duas coisas:

1. **Caso Base (Condição de Parada):** É a resposta mais simples possível, onde a função para de se chamar. Sem isso, seu programa entra em loop infinito e você ganha um belo *Segmentation Fault* (ou *Stack Overflow*).
2. **Passo Recursivo:** É onde a mágica acontece. A função chama a si mesma, mas sempre passando um subproblema **menor**, aproximando-se do caso base.

O Clássico Exemplo do Fatorial

Lembra do fatorial na matemática? $5! = 5 \times 4 \times 3 \times 2 \times 1$. Repare que $5!$ é a mesma coisa que $5 \times 4!$. E $4!$ é $4 \times 3!$.

Veja como escrevemos isso em C:

```
#include <stdio.h>

int fatorial(int n) {
    // 1. Caso Base: o fatorial de 0 ou 1 é sempre 1
    if (n <= 1) {
        return 1;
    }

    // 2. Passo Recursivo: n * fatorial de (n - 1)
    return n * fatorial(n - 1);
}
```

```
int main() {  
    int resultado = fatorial(3);  
    printf("O fatorial de 3 eh: %d\n", resultado);  
    return 0;  
}
```

O que acontece na Memória (A Pilha de Execução)

Quando você chama `fatorial(3)`, o C não consegue resolver o retorno imediatamente porque ele precisa do resultado de `fatorial(2)`. Então, o computador "pausa" a execução atual e empilha uma nova chamada.

O fluxo real na memória funciona exatamente assim:

1. `fatorial(3)` entra na pilha → espera pelo `fatorial(2)`
2. `fatorial(2)` entra na pilha → espera pelo `fatorial(1)`
3. `fatorial(1)` entra na pilha → **Cai no Caso Base!** Retorna 1.
4. A pilha começa a desempilhar resolvendo as contas de trás para frente:
 - a. `fatorial(2)` recebe 1 e faz: $2 \times 1 = 2$
 - b. `fatorial(3)` recebe 2 e faz: $3 \times 2 = 6$
5. O programa finalmente retorna 6.

Resumo

A recursão é uma técnica onde uma função chama a si mesma para resolver subproblemas menores de um problema maior. Para funcionar corretamente e evitar loops infinitos, ela exige obrigatoriamente um caso base que interrompa as chamadas e um passo recursivo que aproxime o cálculo dessa parada. Na prática, o sistema operacional gerencia essas chamadas empilhando-as na memória e resolvendo-as de trás para frente assim que o caso base é atingido.

- **Comparação das respostas:**

As duas respostas explicam corretamente o conceito de recursão, mas a resposta do prompt simples apresenta diretamente a Definição e usa o exemplo intuitivo das bonecas russas, enquanto a resposta do prompt mais elaborado é mais completa e estruturada, incluindo um exemplo prático em C (cálculo do fatorial) com comentários indicando o caso base e o caso recursivo exemplos de código ou detalhes de execução.

Parte 3 - Geração de Código:

- **Prompt utilizado:**

Escreva um programa que leia números digitados pelo usuário e, ao final mostre a média, o maior e o menor valor digitados.

- **Código gerado:**

Inicializa as variáveis

```
soma = 0 contador = 0 maior = None menor = None

print("Digite os números (ou digite 0 para encerrar):")

while True: numero = float(input("Digite um número: "))

# Condição de parada
if numero == 0:
    break

# Somando e contando para a média
soma += numero
contador += 1

# Definindo o maior e o menor valor
if maior == None or numero > maior:
    maior = numero
if menor == None or numero < menor:
    menor = numero
```

Exibição dos resultados (só calcula se pelo menos um número válido foi digitado)

```
if contador > 0: media = soma / contador print("\n--- Resultados ---") print(f"Quantidade de números digitados: {contador}") print(f"Média dos valores: {media:.2f}") print(f"Maior valor digitado: {maior}") print(f"Menor valor digitado: {menor}") else: print("\nNenhum número válido foi digitado.")
```

- **Explicação do código**

- **while True:** Cria um loop infinito que só é interrompido (break) quando o usuário digita 0.
- **maior = None e menor = None:** Começam vazios. No primeiro número que você digita, o programa entende que esse único número é, ao mesmo tempo, o maior e o menor até o momento.
- **f"{media:.2f}":** Formata a média para exibir apenas 2 casas decimais, evitando que apareça um número gigante na tela caso a divisão não seja exata.

- **Teste 1:**

Digite os números (ou digite 0 para encerrar):

Digite um número: 5

Digite um número: 4

Digite um número: 3

Digite um número: 2

Digite um número: 1

Digite um número: 0

--- Resultados ---

Quantidade de números digitados: 5

Média dos valores: 3.00

Maior valor digitado: 5.0

Menor valor digitado: 1.0

- **Teste 2:**

Digite os números (ou digite 0 para encerrar):

Digite um número: 10

Digite um número: 15

Digite um número: 20

Digite um número: 0

--- Resultados ---

Quantidade de números digitados: 3

Média dos valores: 15.00

Maior valor digitado: 20.0

Menor valor digitado: 10.0

- **Análise Crítica:**

O código gerado é correto e resolve o problema proposto. A lógica de inicializar "maior" e "menor" como None e atualizá-los a cada novo número é uma solução comum em Python. No entanto, por o prompt não explicitar a linguagem, no caso de um estudante de uma disciplina baseada em C já não seria uma resposta interessante. Além disso, o LLM resolveu por conta própria usar o valor 0 como parada, o que pode ser ruim para quem deseja considerar também o número 0 como um possível valor.

Parte 4 – Melhorando o Prompt:

- **Prompt utilizado:**

Escreva um programa em C que leia números inteiros digitados pelo usuário, um por vez, terminando quando o usuário digitar -1. Ao final, mostre a média, o maior e o menor valor digitados. Não utilize nenhuma biblioteca além da stdio.h Lembre-se de adicionar comentários em todas etapas.

- **Código gerado:**

```
#include <stdio.h>
```

```
int main() {  
    // Declaração das variáveis  
    int numero;  
    int soma = 0;  
    int contador = 0;  
    int maior, menor;  
    float media;  
  
    printf("Digite números inteiros (ou -1 para encerrar):\n");  
  
    // Loop infinito que será interrompido quando o usuário digitar -1  
    while (1) {  
        printf("Digite um número: ");  
        scanf("%d", &numero);  
  
        // Verifica se o número digitado é a condição de parada  
        if (numero == -1) {  
            break; // Sai do loop imediatamente  
        }  
    }  
}
```

```

// Se for o primeiro número digitado, ele será o maior e o menor inicial
if (contador == 0) {
    maior = numero;
    menor = numero;
} else {
    // Atualiza o maior valor se o número atual for maior que o guardado
    if (numero > maior) {
        maior = numero;
    }
    // Atualiza o menor valor se o número atual for menor que o guardado
    if (numero < menor) {
        menor = numero;
    }
}

// Atualiza a soma acumulada e a quantidade de números lidos
soma += numero;
contador++;
}

// Verifica se o usuário digitou pelo menos um número válido antes de
testar o -1
if (contador > 0) {
    // Calcula a média (o 'float' garante que a divisão não seja apenas
    inteira)
    media = (float)soma / contador;

    // Exibe os resultados na tela
    printf("\n--- Resultados ---\n");
    printf("Quantidade de números digitados: %d\n", contador);
    printf("Média dos valores: %.2f\n", media);
    printf("Maior valor digitado: %d\n", maior);
    printf("Menor valor digitado: %d\n", menor);
} else {
    // Caso o usuário tenha digitado -1 logo de primeira
    printf("\nNenhum número válido foi digitado.\n");
}

return 0;
}

```

- **Controle de Maior/Menor:** A lógica utiliza o `contador == 0` para garantir que o primeiro número válido digitado inicialize as variáveis

maior e menor. Isso evita o erro comum de inicializá-las com 0 (o que quebraria a lógica caso o usuário só digitasse números negativos, por exemplo).

- **Casting (float):** Na linha do cálculo da média, usamos (float) soma para converter temporariamente a soma inteira em ponto flutuante. Isso garante que o C faça uma divisão decimal exata (ex: $5/2=2.5$ e não 2).
- **Apenas stdio.h:** Toda a entrada (scanf) e saída (printf) de dados foi resolvida utilizando apenas as funções nativas da biblioteca padrão solicitada.

- **Comparação com a versão anterior:**

Aqui a LLM produziu um código na linguagem requerida, com comentários adequados no código, e usando o valor dado como valor de parada. É um código muito mais adequado para a disciplina, e moldado de forma mais específica às necessidades do usuário.

Parte 5 – Verificação crítica:

Resposta escolhida para a análise: Código gerado na parte 3

- **1. A resposta estava correta?**

Sim, apesar de ser ampla e não específica em alguma linguagem para algum objetivo.

- **2. Havia informações incompletas ou incorretas?**

- Não havia erros de lógica, as algumas escolhas no código foram determinadas pela LLM e podem não se encaixar em certos casos

- **3. Como a resposta foi verificada?**

Compilando e rodando o código no VSCode e conferindo a saída

- **4. A LLM demonstrou excesso de confiança?**

Sim, a solução foi apresentada de forma complexa e definitiva, e ela não se importou em destacar que era um prompt vago e possivelmente a resposta não se encaixaria para muitos casos.

- **5. Como o prompt poderia ser melhorado?**

Como eu fiz na parte 4, explicitando a linguagem requerida, dizendo quais devem ser as determinações no código (-1 para encerrar o loop, por exemplo), definir as entradas inválidas.

Uso responsável das LLMs:

1. Por que não devemos copiar respostas sem revisão?

Por que, como ficou evidente na Parte 3, a LLM pode tomar decisões por si só que talvez não correspondam ao que o usuário realmente necessita.

2. Por que é importante informar os prompts utilizados?

Informar os prompts permite que outras pessoas entendam exatamente o que foi pedido à LLM e o motivo da resposta sair daquela forma. Assim é possível avaliar o quanto o resultado é afetado pelo prompt.

3. Em quais situações as LLMs ajudam no aprendizado?

Ajudam como auxílio para entender conceitos complexos e como assistentes, utilizando de forma responsável.

4. Em quais situações elas podem atrapalhar?

Podem atrapalhar quando o usuário não se preocupa em construir um bom prompt ou apenas copia o que a LLM retorna sem verificar por conta própria.

5. O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?

Significa usar a LLM para esclarecer dúvidas, obter exemplos, fazer testes, e não apenas pedindo para que ela resolva todo problema e copiando sem raciocinar.

Reflexão sobre o Uso da IA Generativa:

1. O que você aprendeu sobre interação com LLMs?

Aprendi que a especificidade do prompt tem um impacto direto na qualidade e adequação da resposta. Quando o prompt deixa espaço para interpretação, a LLM preenche com determinações que não podem ser as mais adequadas.

2. Como a qualidade do prompt influenciou os resultados?

O mesmo problema (ler números e calcular média, maior e menor), com um prompt vago, resultou em um código em Python sem comentários e com um valor de parada escolhido arbitrariamente. Já com um prompt detalhado, especificando linguagem, restrição de bibliotecas (stdio.h), valor de parada, e exigência de comentários, o resultado foi um código muito mais alinhado e específico para alguma tarefa.

3. Em quais situações a ferramenta foi mais útil?

Na Parte 2 e 4, onde foram utilizados prompts mais elaborados que garantiram um resultado mais certo e adequado.

4. Em quais situações foi necessário verificar ou corrigir respostas?

Na parte 3, onde o prompt simples fez um código que embora correto não seria adequado em contextos específicos, surgindo a necessidade da criação de um novo prompt na parte 4.

5. Você utilizaria essa ferramenta em disciplinas futuras? Por quê?

Sim, principalmente como apoio para entender conceitos complexos e para gerar exemplos práticos. Porém, com responsabilidade e utilizando de bons prompts, se não como demonstrado no relatório surgirão apenas respostas vagas que podem não potencializar meus estudos.

